

Es hora de que pongas en práctica todo lo aprendido. 🧐

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😊

¡Manos a la obra!

1. Desafío 🎯

Implementar un modelo de clasificación supervisada con MLlib.

Tareas

- Crear una sesión de Spark.
- Cargar un conjunto de datos de muestra (puede ser el dataset Iris o un archivo CSV simulado).
- Preparar los datos utilizando VectorAssembler y, si es necesario, StringIndexer.
- Dividir los datos en conjunto de entrenamiento y prueba.
- Entrenar un modelo de Regresión Logística utilizando MLlib.
- Realizar predicciones sobre el conjunto de prueba.
- Evaluar el modelo utilizando métricas como Accuracy o ROC-AUC.

2. ¿Dónde se lleva a cabo? 🏠

Visual Studio Code

3. Tiempo de dedicación ⌚

2 horas

4. Recursos 🛠️

- <https://spark.apache.org/docs/latest/ml-guide.html>
- <https://certidevs.com/tutorial-pyspark-regresion-mllib-machine-learning>
- https://www.datacamp.com/es/tutorial/pyspark-tutorial-getting-started-with-pyspark?utm_source=chatgpt.com

5. Plus +

✓ Como desafío adicional, podés comparar el rendimiento de la Regresión Logística con otro modelo supervisado como Random Forest y analizar las diferencias.

6. ⚠ Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío

1. Importar librerías necesarias

```
from pyspark.sql import SparkSession  
  
from pyspark.ml.feature import VectorAssembler, StringIndexer  
  
from pyspark.ml.classification import LogisticRegression  
  
from pyspark.ml.evaluation import BinaryClassificationEvaluator  
  
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
```

2. Crear sesión Spark

```
spark = SparkSession.builder \  
    .appName("EjercicioMLlib") \  
    .master("local[*]") \  
    .getOrCreate()
```

3. Cargar un dataset de ejemplo (puede ser reemplazado por un CSV real)

```
data = spark.createDataFrame([  
    (0, 25, 50000, 0),  
    (1, 45, 100000, 1),  
    (2, 35, 60000, 0),  
    (3, 50, 120000, 1),  
    (4, 23, 40000, 0),  
    (5, 40, 90000, 1)  
], ["id", "edad", "ingreso", "abandono"])
```

4. Preparar los datos

```
assembler = VectorAssembler(  
    inputCols=["edad", "ingreso"],  
    outputCol="features"  
)  
  
data = assembler.transform(data).select("features", "abandono")
```

5. Dividir los datos en entrenamiento y prueba

```
train_data, test_data = data.randomSplit([0.8, 0.2], seed=1234)
```

6. Definir y entrenar el modelo

```
lr = LogisticRegression(featuresCol="features", labelCol="abandono")  
  
modelo = lr.fit(train_data)
```

7. Realizar predicciones

```
predicciones = modelo.transform(test_data)  
  
predicciones.select("features", "abandono", "prediction",  
    "probability").show()
```

8. Evaluar el modelo

```
evaluator = BinaryClassificationEvaluator(labelCol="abandono",  
    metricName="areaUnderROC")  
  
roc_auc = evaluator.evaluate(predicciones)
```

```
print(f"Área bajo la curva ROC: {roc_auc}")
```

```
# 9. (Opcional) Ajuste de hiperparámetros
```

```
paramGrid = ParamGridBuilder() \  
.addGrid(lr.regParam, [0.01, 0.1, 0.5]) \  
.addGrid(lr.maxIter, [10, 20]) \  
.build()
```

```
crossval = CrossValidator(  
    estimator=lr,  
    estimatorParamMaps=paramGrid,  
    evaluator=evaluator,  
    numFolds=3  
)
```

```
cv_model = crossval.fit(train_data)  
mejor_modelo = cv_model.bestModel
```

```
# Evaluación del mejor modelo
```

```
predicciones_cv = mejor_modelo.transform(test_data)  
roc_auc_cv = evaluator.evaluate(predicciones_cv)  
print(f"Área bajo la curva ROC (Cross Validation): {roc_auc_cv}")
```

```
# 10. Detener sesión Spark
```

```
spark.stop()
```

